

Full-Stack Internship Assignment: Invoice Management Dashboard

Overview

You are provided with a seed dataset (`seed-data.json`) containing **2,000 invoice records** spanning **61 unique customers**, where each customer is associated with exactly one company. Your task is to build a full-stack invoice management application that ingests this data and exposes it through a performant API and an interactive dashboard.

Tech Stack (required)

- **Frontend:** React.js
- **Backend:** Node.js (Express or Nest, your choice)
- **Database:** MongoDB with Mongoosejs

Fields definition in the seed dataset

Each record has the following shape:

invoiceId	string	e.g. "INV-6598015"
customer	string	
company	string	one company per customer (1:1)
amount	number	pre-tax base amount
taxRate	number	one of 0, 3, 5, 18, 28 (percent)
tax	number	= amount × taxRate / 100
total	number	= amount + tax
status	string	Sent Unpaid Overdue Paid Void Draft
issueDate	string	ISO date (YYYY-MM-DD)
dueDate	string	ISO date (YYYY-MM-DD)

Requirements

1. Models and Schema Definition

Based on the field definition above and the requirements mentioned in 3; define appropriate data models

and respective schema for each model. Refer [Monogoose Schema](#).

Model the data sensibly and explain your modeling choice in the README

2. Data Ingestion

Write a seed script that loads `data-generator.json` into MongoDB. Provide steps to run the script in the README

3. Dashboard Requirements

You can take help of the low fidelity wirframes attached.

- A paginated, sortable(amount and due date) , filterable (status, customer, issueDate/issueDate range, dueDate/dueDate range) invoice table wired to the API.
- A summary view of top 5 customers.
- A **customer profile view** that, given a customer, shows their company, their full invoice history, and summary metrics for that customer.
- A **create and edit invoice form**.

4. Deliverables

- A Git repository containing the backend, frontend code
- Installation steps
- Seed script
- A README covering setup, run instructions, data modeling rationale, and any assumptions.

Stretch Goals (optional)

- Containerize with Docker Compose (Mongo + API + FE).
- Unit/integration tests